

FONKSİYONLAR

Fonksiyon Nedir?

Bir Python programı, genel olarak, tek başına tüm görevleri yerine getiren tek kod içeren dosyadan veya kod bloğundan oluşmaz. Bu durumda ana problemin büyüklüğüne göre kod bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani **böl ve yönet** (**divide and conquer**) tekniği uygulanır.

💡 Büyük bir problemin toptan çözülmesi, problemin parçalar halinde çözülmesinden her zaman daha yorucu ve zordur.

Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna **yukarıdan aşağıya** (**top down**) yaklaşım adı verilir.

Yapısal Programlama mantığında çözülecek problem genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Python dili de fonksiyonları destekler. Ayrıca birbiriyle ilişkili fonksiyon, değişken ve nesneler bir modül olarak tasarlanır.

Bir Python modülü, Python tanımları ve ifadeleri içeren bir dosyadır. Bir modül, fonksiyonları, sınıfları ve değişkenleri tanımlayabilir. Bir modül ayrıca çalıştırılabilir kod da içerebilir. Birçok Python modülü vardır.

Python dilinde fonksiyonlar;

- Fonksiyonlar, belirli bir görevi gerçekleştiren bir kod bloğudur.
- Parametrelili alabilir, girdilerle ya da parametresiz bir şeyler yapar ve ardından cevabı verebilir. Kısaca bilgiyi fonksiyona argümanlar ile aktarırız.
- Her fonksiyonun bir kimliği vardır. **Değişken kimliklendirme kuralları fonksiyonlar için de geçerlidir!**
- Bir fonksiyon program içerisinde istenildiği kadar farklı yerlerden ulaşılarak çalıştırılabilir ya da çağrılabilir. Bir başka deyişle tekrar kullanılabilir.
- Bir fonksiyon, çağıran koda bir değer döndürebilir.
- Bir fonksiyon, bir başka fonksiyondan çağrılabilir.

İki tip fonksiyon bulunmaktadır; başlık dosyalarında bize sunulan **hazır fonksiyonlar** ve programcının tanımlayacağı **kullanıcı tanımlı fonksiyonlar** (**user defined function**).

Hazır Fonksiyonlar

Programcıların kullanmaları için, önceden yazılmış `print()` ve `input()` gibi hazır fonksiyonlar Python dilinde **dahili fonksiyon** (**built-in function**) olarak sunulur. Bu fonksiyonlar `builtins` modülünde bulunur. Python yorumlayıcısı, her zaman kullanılabilir olan bu modüle ait bir dizi yerleşik fonksiyona ve veri tiplerine sahiptir. Bunlar `dir(__builtins__)` talimatı ile görüntülenebilir¹³.

```
>>> dir( __builtins__ )
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BaseExceptionGroup',
'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError',
'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError',
'DeprecationWarning', 'EOFError', 'Ellipsis', 'EncodingWarning', 'EnvironmentError',
'Exception', 'ExceptionGroup', 'False', 'FileExistsError', 'FileNotFoundError',
'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError', 'IsADirectoryError',
'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError', 'ModuleNotFoundError',
```

¹³ <https://docs.python.org/3/library/functions.html>

```
'NameError', 'None', 'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError',
'OverflowError', 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError',
'PythonFinalizationError', 'RecursionError', 'ReferenceError', 'ResourceWarning',
'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration', 'StopIteration', 'SyntaxError',
'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError',
'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError',
'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError', 'Warning',
'WindowsError', 'ZeroDivisionError', '_', '_IncompleteInputError', '__build_class__',
'__debug__', '__doc__', '__import__', '__loader__', '__name__', '__package__', '__spec__',
'abs', 'aiter', 'all', 'anext', 'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray',
'bytes', 'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits',
'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit', 'filter', 'float',
'format', 'frozenset', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input',
'int', 'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max',
'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property',
'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted',
'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']
```

Bu fonksiyonların bir kısmı anlatılmıştır. Yeri geldikçe diğerleri anlatılacaktır. Bu fonksiyonlara ilişkin yardım dokümanına `help` komutu ile aşağıdaki şekilde ulaşılabilir.

```
>>> help
Welcome to Python 3.13's help utility! If this is your first time using
Python, you should definitely check out the tutorial at
https://docs.python.org/3.13/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules. To get a list of available
modules, keywords, symbols, or topics, enter "modules", "keywords",
"symbols", or "topics".

Each module also comes with a one-line summary of what it does; to list
the modules whose name or summary contain a given string such as "spam",
enter "modules spam".

To quit this help utility and return to the interpreter,
enter "q", "quit" or "exit".

help> id
Help on built-in function id in module builtins:

id(obj, /)
    Return the identity of an object.

    This is guaranteed to be unique among simultaneously existing objects.
    (CPython uses the object's memory address.)

help> q

You are now leaving help and returning to the Python interpreter.
If you want to ask for help on a particular object directly from the
interpreter, you can type "help(object)". Executing "help('string')"
has the same effect as typing a particular string at the help> prompt.
>>>
```

Benzer şekilde `math` modülünde ise yaygın matematik fonksiyonları ve sabitleri bulunur. Her Python programı için ön tanımlı olan `builtin` modülü dışındaki modülleri koda dahil etmek için `import` kelimesi kullanılır. Örnek olarak aşağıda `math` modülü verilmiştir.

```
# ÜÇGENİN ALANI - HERON FORMÜLÜ
import math
kenar1str=input('Üçgenin birinci kenarını Giriniz:')
kenar1=float(kenar1str)
```

```
kenar2str=input('Üçgenin ikinci kenarını Giriniz:')
kenar2=float(kenar2str)
kenar3str=input('Üçgenin üçüncü kenarını Giriniz:')
kenar3=float(kenar3str)
if (kenar1+kenar2)<=kenar3 or (kenar1+kenar3)<=kenar2 or (kenar2+kenar3)<=kenar1:
    print('Böyle bir üçgen olamaz!')
else:
    kenarlarToplamınınYarısı=(kenar1+kenar2+kenar3)/2
    alan=math.sqrt( kenarlarToplamınınYarısı*
                    (kenarlarToplamınınYarısı-kenar1)*
                    (kenarlarToplamınınYarısı-kenar2)*
                    (kenarlarToplamınınYarısı-kenar3) )
    print('Üçgenin Alanı:',alan)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Üçgenin birinci kenarını Giriniz:3.0
Üçgenin ikinci kenarını Giriniz:4.0
Üçgenin üçüncü kenarını Giriniz:5.0
Üçgenin Alanı: 6.0
```

Matematik modülü fonksiyon ve sabitleri aşağıda talimatla öğrenilebilir¹⁴. Benzer şekilde **aktarılan** (**import**) edilerek tüm modül fonksiyon, sınıf ve sabitleri listelenebilir.

```
>>> import math
>>> dir (math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin',
'asinh', 'atan', 'atan2', 'atanh', 'cbrt', 'ceil', 'comb', 'copysign', 'cos', 'cosh',
'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'exp2', 'expm1', 'fabs', 'factorial', 'floor',
'fma', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite',
'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf',
'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh',
'sqrt', 'sumprod', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
```

Aynı modüldeki sabitlerin kullanımına örnek olarak yarıçapı girilen bir dairenin çevre ve alan hesabı verilebilir.

```
import math
yarıçapstr=input('Yarıçapı Giriniz:')
yarıçap=float(yarıçapstr)
print('Dairenin Çevresi:',2*math.pi*yarıçap)
print('Dairenin Alanı:',math.pi*yarıçap*yarıçap)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Yarıçapı Giriniz:12.0
Dairenin Çevresi: 75.39822368615503
Dairenin Alanı: 452.3893421169302
```

Yukarıdaki örnekte `math.pi` sabitini kullanmak için tüm `math` modülü **aktarılmıştır** (**import**). Eğer bir modülden sadece belirlenen bir üye aktarılacak ise `from` talimatı kullanılır. Verilen örnek bu kapsamda aşağıdaki şekilde yazılabilir.

```
from math import pi
yarıçapstr=input('Yarıçapı Giriniz:')
yarıçap=float(yarıçapstr)
print('Dairenin Çevresi:',2*pi*yarıçap)
print('Dairenin Alanı:',pi*yarıçap*yarıçap)
```

Dahili modül yanında, Python dili birçok modülü bize standart olarak sunar¹⁵.

¹⁴ <https://docs.python.org/3/library/math.html#math.sqrt>

¹⁵ <https://docs.python.org/3/library/index.html>

Kullanıcı Tanımlı Fonksiyonlar

Python dilinde dahili olan fonksiyonlar ya da `math` modülündeki fonksiyonlar gibi kendi ihtiyaçlarımızı karşılayan yeniden kullanılabilir fonksiyonlar tanımlayabilir ve modüller oluşturabiliriz.

Fonksiyon Tanımlama

Bir Python fonksiyonu aşağıdaki kurallar ile tanımlanır;

1. Fonksiyon blokları `def` anahtar sözcüğüyle başlar, bunu fonksiyon adı ve parantezler `()` izler.
2. Herhangi bir giriş parametresi veya argümanı bu parantezlerin içine yerleştirilmelidir. Parametreleri bu parantezlerin içine de tanımlayabilirsiniz.
3. Her fonksiyon içindeki kod bloğu iki nokta üst üste `:` ile başlar ve girintilidir.
4. Fonksiyon bloğunun ilk satırında fonksiyonun ne yaptığını kısaca anlatan bir dokümantasyon metni **dizgi** olarak (**docstring**) yer alabilir. Bu **docstring**, modülleri, sınıfları, fonksiyonları ve yöntemleri belgelemek için kullanılan çok satırlı bir yorumdur. Bileşenin ilk talimatı olmalıdır.
5. Fonksiyon bloğu içinde `return [expression]` ifadesi bir fonksiyondan çıkar ve isteğe bağlı olarak çağırana bir ifade geri iletebiliriz. Argümanı olmayan bir `return` ifadesi `return None` ile aynıdır.

Fonksiyon tanımına ilişkin sözde kod aşağıda verilmiştir.

```
def fonksiyon-kimliği( parametreler ):
    "Fonksiyon Tanımı (docstring)"
    # fonksiyon gövdesi
    return [expression]
```

Aşağıda fonksiyon tanımlamalarına ve bu fonksiyonların çağırılmasına kısa bir örnek verilmiştir.

```
#Fonksiyon Tanımları
def ikiSayıTopla(x,y): # Parametrelili fonksiyon
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def sayıYaz(sayı): # Geri değer döndürmeyen fonksiyon
    print(sayı)
    return # yazılmayabilir
def sayıOku(): # Parametresiz fonksiyon
    str=input('Bir Sayı Giriniz:')
    return float(str)
#Artık Tanımlanmış Fonksiyonları Çağırabiliriz
sayı1=sayıOku() # sayıOku fonksiyonu çağırılıyor
sayı2=sayıOku() # sayıOku fonksiyonu tekrar çağırılıyor
sonuc=ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağırılıyor
sayıYaz(sonuc) # sayıYaz fonksiyonu çağırılıyor
print(ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Bir Sayı Giriniz:10
Bir Sayı Giriniz:20
30.0
Bu Fonksiyon Parametreleri Toplar
```

Dokümantasyon metni (**docstring**) için günümüzde PEP 257 standardı kullanılmaktadır. Aşağıda örneği verilmiştir;

```
def toplam(a, b) :
    """
    İki sayıyı toplar.

    Args:
        a : İlk sayı
        b : İkinci sayı
```

```

Returns:
    İki sayının toplamı

Examples:
    >>> toplam(2, 3)
    5
"""
return a + b

```

Fonksiyonlara Parametre Geçirme

Bir fonksiyonun tanımlanması sırasında parantez içinde bildirilen değişkenlerin listesi **resmi argümanlardır** (**formal argument**) ve bu argümanlar **konumsal argümanlar** (**positional argument**) olarak da bilinir. Bir fonksiyon istenildiği kadar resmi argümanla tanımlanabilir. Fonksiyonlara geçirilen parametreler tanımlı ise çağrılırken mutlaka argüman verilmelidir.

```

def negatif(x):
    "Bu fonksiyon parametresiz çağrılmaz."
    return -x;
sonuc=negatif(-1)
print(sonuc) #1
# sonuc=negatif() hata verir.
a=-1;
sonuc=negatif(a) #bir değişken de argüman olarak verilebilir
print(sonuc) #1

```

Fonksiyonlara parametre geçirirken bazı parametreler **ön tanımlı argümanlara** (**default argument**) sahip olabilir.

```

# Fonksiyon Tanımı
def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
# Artık Fonksiyonu Kullanabiliriz
sonuc=ikiSayıTopla(10) #ikinciargman 1 kabul edildi
print(sonuc) #11

```

Tüm parametreler için de ön tanımlı argüman belirlenebilir.

```

def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def ikinciSayıyıÇıkar(x=10,y=1):
    "Bu Fonksiyon Birinci Parametreden İkincisini Çıkarır"
    fark=x-y
    return fark
sonuc=ikiSayıTopla(10) #ikinciargman 1 kabul edildi
print(sonuc) #11
sonuc=ikinciSayıyıÇıkar() #birinci argüman 10, ikincisi 1 kabul edildi
print(sonuc) #9

```

Parametre ismi verilerek argümanlar fonksiyona geçiriliyorsa bu argümanlara **anahtar kelime argümanlar** (**keyword argument**) adı verilir;

```

def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
sonuc=ikiSayıTopla(x=10) #ikinciargman 1 kabul edildi
print(sonuc) #11

```

```
sonuc=ikiSayıTopla(x=10, y=20)
print(sonuc) #30
sonuc=ikiSayıTopla(y=10, x=20)
print(sonuc) #30
```

Fonksiyonlardaki argüman listesindeki değişkenleri değer geçirmek için anahtar kelime olarak kullanabilirsiniz. **Anahtar kelime argümanlarının** (keyword argument) kullanımı isteğe bağlıdır. Ancak, fonksiyonun argümanları yalnızca anahtar kelimeyle kabul etmesini zorlayabilirsiniz. Bunun için argümanlar listesinden önce bir yıldız işareti (*) koymalısınız. Dahili print() fonksiyonu yalnızca anahtar sözcük argümanlarına bir örnektir. Parantez içinde yazdırılacak ifadelerin listesini verebilirsiniz. Yazdırılan değerler varsayılan olarak bir boşlukla ayrılır. sep argümanını kullanarak bunun yerine başka herhangi bir ayırma karakteri belirtebilirsiniz.

```
print (1,2,3,4,5, sep='-') #1-2-3-4-5
print ("Bir","İki","Üç","Dört", sep=",") #Bir,İki,Üç,Dört
```

Bir argümanı yalnızca anahtar kelimedenden oluşan bir argüman yapmak için, kullanıcı tanımlı fonksiyonu oluştururken argümanın önüne yıldız işareti (*) koyulur.

```
from math import pow
def kuvvet(taban,*, üs):
    sonuc = pow(taban,üs)
    return sonuc

hesap = kuvvet(3, üs=4)
print(hesap) #81.0
```

Fonksiyon, bir veya daha fazla argümanın anahtar sözcüklerle değerlerini kabul edemeyecek şekilde tanımlanabilir. Bu tür argümanlara **yalnızca konumsal argümanlar** (positional-only argument) denir. Bir argümanı yalnızca konumsal yapmak için bölü (/) sembolünü kullanılır. Bu sembolden önceki tüm argümanlar yalnızca konumsal olarak ele alınacaktır.

```
from math import pow
def kuvvet(taban, üs, /):
    sonuc = pow(taban,üs) #birinci parametre taban, ikinci parametre üs kabul edilir.
    return sonuc

hesap = kuvvet(3,4)
print(hesap) #81.0
hesap =kuvvet(2,8)
print (hesap) #256.0
```

Bir başka örnek;

```
def hepsiniYaz(x,/,y,*,z): # birinci parametre her zaman ilk sırada,
                           # z de anahtar kelime zorunlu
    print(x,y,z)
    return
hepsiniYaz(100,10,z=20) # 100 10 20
hepsiniYaz(100,y=10,z=20) # 100 10 20
hepsiniYaz(100,z=20,y=10) # 100 10 20
```

Değişken sayıda argüman kabul edebilen bir fonksiyon tanımlamak isteyebiliriz. Bu argümanlara **keyfi sayıda argüman** (arbitrary argument) adı verilir. Ayrıca, değişken sayıda argüman konumsal veya anahtar kelime argümanları olabilir.

```
def hepsinitopla(*args):
    s=0
    for x in args:
        s=s+x
    return s
sonuç = hepsinitopla(3,5,7,9)
print (sonuç) #24
```

```
sonuç = hepsinitopla(150,300,450)
print (sonuç) #900
```

Keyfi sayıda argüman yanında zorunlu argüman da tanımlanabilir;

```
def katsayıİleÇarpTopla(katsayı, *args):
    s=0
    for x in args:
        s=s+x*katsayı
    return s
sonuç = katsayıİleÇarpTopla(10,3,5,7,9)
print (sonuç) #240
sonuç = katsayıİleÇarpTopla(3,150,300,450)
print (sonuç) #2700
```

Keyfi sayıda anahtar kelime argümanı da tanımlayabiliriz;

```
def keyfianahtarkelimeargüman(**kwargs):
    for k,v in kwargs.items():
        print ("{}:{}".format(k,v))
keyfianahtarkelimeargüman(adı='İlhan',soyadı='ÖZKAN',yaş=50,boy=1.8)
```

Bu programı çalıştırdığımızda aşağıdaki çıktı elde edilir;

```
adı:İlhan
soyadı:ÖZKAN
yaş:50
boy:1.8
```

Fonksiyon Açıklamaları

Python dilinde veri tipleri dinamik olarak belirlenir. Bazı fonksiyonlarda işlenecek veriler ile geri döndürülecek değerler için veri tipi belirlenmesi gerekir. Bunun için fonksiyon **açıklama (annotation)** özelliği kullanılır. Bir fonksiyon tanımında bildirilen argümanlar ve ayrıca dönüş veri tipi hakkında ek açıklayıcı meta veriler eklemenize olanak tanır.

```
def topla(x, y): #hiçbir açıklaması olmayan fonksiyon
    geçici = x+y
    return geçici
print(topla(10,20))
print(topla.__annotations__)
```

Dönüş veri tipi için de açıklama verebilirsiniz. Fonksiyon parantezinden sonra ve iki nokta üst üste sembolünden önce, eksi ve büyük karakterinden oluşan bir ok (->) koyulur. Ve dönüş tipi belirtilir.

```
def topla(x, y):
    geçici = x+y
    return geçici
print(topla(10,20)) #30
print(topla.__annotations__) #{}
def intTopla(x:int, y:int): #parametreleri int olan topla fonksiyonu
    geçici = x+y
    return geçici
print(intTopla(10,20)) #30
print(intTopla.__annotations__) #{'x': <class 'int'>, 'y': <class 'int'>}
def intToplaintDöndür(x:int, y:int) -> int: #parametreleri ve geri döndürdüğü değer
                                         #int olan topla fonksiyonu
    geçici = x+y
    return geçici
print(intToplaintDöndür(10,20)) #30
print(intToplaintDöndür.__annotations__)
#{'x': <class 'int'>, 'y': <class 'int'>, 'return': <class 'int'>}
```

Değişken Kapsamı

Bir değişkenin **kapsamı** (**scope**), değişkenin kullanıcı tarafından erişilebilir olduğu yerler olarak tanımlanır.

Bir **yerel değişken** (**local variable**), belirli bir fonksiyon veya kod bloğu içinde tanımlanır. Yalnızca tanımlandığı fonksiyon veya blok tarafından erişilebilir ve sınırlı bir kapsamı vardır. Başka bir deyişle, yerel değişkenlerin kapsamı tanımlandıkları fonksiyonla sınırlıdır ve bu fonksiyon dışından bunlara erişmeye çalışmak hataya yol açacaktır.

```
def hepsinitopla(*args):
    s=0 #yerel değişken
    for x in args:
        s=s+x
    print(locals()) # {'args': (10, 20, 30, 40), 's': 100, 'x': 40}
    return s
#fonksiyon dışında s ve x değişkenlerine ulaşılamaz.
sonuc= hepsinitopla (10,20,30,40)
```

Bir **küresel değişkene** (**global variable**) programın herhangi bir bölümünden erişilebilir ve herhangi bir fonksiyon veya kod bloğunun dışında tanımlanır. Herhangi bir blok veya fonksiyona özgü değildir.

```
programcıAdı = 'İlhan ÖZKAN'
uyarlama=2.3
def birFonksiyon():
    print("Programcı Adı:", programcıAdı)
    print("Uyarlama (version):", uyarlama)
    print(locals())
def birBaşkaFonksiyon():
    temp=0
    print("Programcı Adı:", programcıAdı)
    print(temp)
    print(locals())
print(programcıAdı)
birFonksiyon()
birBaşkaFonksiyon()
print(globals())
```

Programın çıktısı aşağıdaki gibi olur;

```
İlhan ÖZKAN
Programcı Adı: İlhan ÖZKAN
Uyarlama (version): 2.3
{}
Programcı Adı: İlhan ÖZKAN
0
{'temp': 0}
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__':
<_frozen_importlib_external.SourceFileLoader object at 0x7ffba8057f80>, '__spec__': None,
'__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, '__file__':
'/home/cg/root/685b2ccdb7180/main.py', '__cached__': None, 'programcıAdı': 'İlhan ÖZKAN',
'uyarlama': 2.3, 'birFonksiyon': <function birFonksiyon at 0x7ffba804a2a0>,
'birBaşkaFonksiyon': <function birBaşkaFonksiyon at 0x7ffba7e4d300>}
```

Özyinelemeli Fonksiyonlar

Özyineleme (**recursion**), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, **yineleme** (**iteration**), **döngüler** (**loop**) veya tekrar yerine geçebilecek çok güçlü bir tekniktir.

Özyinelemeli (recursive) algoritmelerde, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur. Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır.

1. Cevabı kesin olarak bildiğimiz **temel durum (base case)**
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n 'nin faktöriyeli, $n!$ ile gösterilir ve n 'den küçük veya ona eşit olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 3 \cdot 2 \cdot 1$$

Negatif olmayan bir tam sayı olan n 'nin faktöriyeli aynı zamanda n 'nin bir sonraki daha küçük faktöriyelle çarpımına eşittir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \cdot (n - 1)!$$

Örneğin;

$$5! = 5 \cdot 4! = 5 \cdot 4 \cdot 3! = 5 \cdot 4 \cdot 3 \cdot 2! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
def faktöriyel(n):
    if n>=1:
        çarpım=1
        for n in range(2,n+1):
            çarpım=çarpım*n
        return çarpım
    else:
        return 1

faktöriyel(5) #120
faktöriyel(7) #5040
```

Sıfır sayısı da pozitif bir sayıdır ama sonucum sıfır çıkmaması için 0 sayısının faktöriyeli 1 kabul edilir. Bu aslında faktöriyel problemi için **temel durumdur (base case)** ve **boş ürün (empty product)** kuralı olarak bilinir. Kısaca faktöriyel fonksiyonu aşağıdaki şekilde gösterilir;

$$n \in \mathbb{N} \text{ olmak üzere } f(n) = n! = \begin{cases} 1, & n = 0 \\ n \cdot f(n - 1), & n > 0 \end{cases}$$

Özyinelemeli (recursive) olarak ise bu fonksiyon aşağıdaki şekilde kodlanabilir;

```
def faktöriyel(n):
    if n==1:
        return 1 #base case
    else:
        return n*faktöriyel(n-1)
```

Girilen taban ve üs ile kuvvet hesaplama fonksiyonu özyinelemeli olarak aşağıda verilmiştir;

```
def kuvvet(taban,üs):
    if taban==0:
        return 0 #base case
    if üs==0:
        return 1 #base case
    return taban*kuvvet(taban,üs-1)

kuvvet(2,3) #8
kuvvet(3,2) #9
```

Kendi Modülümüzü Oluşturma

Bir modül, tanımları ve ifadeleri içeren içe aktarılabilir bir Python dosyasıdır. Modüller, bir `.py` dosyası oluşturarak oluşturulabilir. Yukarıdaki örneğimizde fonksiyonları `Fonksiyonlar.py` dosyasında yazalım;

```
#Fonksiyonlar.py
"""Bu Modül Okuma Yazma ve Toplama Fonksiyonu Bulunur.""" # Modül Dokümantasyon Metni
def ikiSayıTopla(x,y): # Parametrelili fonksiyon
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def sayıYaz(sayı): # Geri değer döndürmeyen fonksiyon
    print(sayı)
    return #yazılmayabilir
def sayı0ku(): # Parametresiz fonksiyon
    str=input('Bir Sayı Giriniz:')
    return float(str)
```

Bir modüldeki fonksiyonlar, modülün içe aktarılmasıyla (`import`) kullanılabilir. Bu durumda fonksiyonları çağıran kodumuz aşağıdaki şekilde olacak ve aynı çıktıyı verecektir;

```
import Fonksiyonlar
sayı1=Fonksiyonlar.sayı0ku() # sayı0ku fonksiyonu çağrılıyor
sayı2=Fonksiyonlar.sayı0ku() # sayı0ku fonksiyonu tekrar çağrılıyor
sonuc=Fonksiyonlar.ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağrılıyor
Fonksiyonlar.sayıYaz(sonuc) # sayıYaz fonksiyonu çağrılıyor
print(Fonksiyonlar.ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Oluşturduğunuz modüller, bunları içeri aktardığınız (`import`) dosya ile aynı klasörde olmalıdır. Ancak, önceden dahil edilmiş modüllerle birlikte Python `lib` klasörüne de koyabilirsiniz, ama mümkünse bundan kaçınılmalıdır.

Bunların dışında modüllere takma isim de verilebilir;

```
import Fonksiyonlar as Fn
sayı1=Fn.sayı0ku() # sayı0ku fonksiyonu çağrılıyor
sayı2=Fn.sayı0ku() # sayı0ku fonksiyonu tekrar çağrılıyor
sonuc=Fn.ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağrılıyor
Fn.sayıYaz(sonuc) # sayıYaz fonksiyonu çağrılıyor
print(Fn.ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Modüle ilişkin dokümantasyon metnini öğrenmek için aşağıdaki kod yazılabilir;

```
import Fonksiyonlar as Fn
print( Fn.__doc__ )
```

Örneğin program akışı aşağıdaki gibi olacaktır.

Bu Modül Okuma Yazma ve Toplama Fonksiyonu Bulunur.

Paket (Package) yönetimi

Python ilinde `modül` (`module`), `.py` uzantılı sınıflar, fonksiyonlar gibi nesneler içeren bir Python dosyasıdır. Python dilinde `paketler` (`package`), bir veya daha fazla modül dosyası içeren bir klasördür ve bu klasör `__init__.py` adlı özel bir dosya içerir. Bu dosya boş olabilir ancak paket listesini içerebilir.

Örnek olarak `hesap` adı bir Python paketi aşağıda verilmiştir;

<pre>D:\ ├── PythonKlasoru\ │ ├── hesap\ │ │ ├── __init__.py │ │ ├── polinom.py │ │ └── alan.py</pre>	<pre>→ → Python Betik Dosyalarının Bulunduğu Klasör → Paket Adını Belirleyen Klasör → Paket Listesini İçeren Özel Dosya → paket içindeki betik dosyası → paket içindeki bir başka betik dosyası</pre>
---	---



alanOrnek.py
polinomOrnek.py

→ Paket dışındaki betik dosyası
→ Paket dışındaki bir başka betik dosyası

Polinom modülü içeriği aşağıdaki gibidir;

```
# polinom.py
"""Bu Modül aşağıda terim katsayıları verilen polinom değerini hesaplar."""
def polinomDegeri(x,*katsayilar):
    """İlk Parametre x, ikinci parametre sabit terim katsayısı ..."""
    deger=0.0;
    indis=0
    for katsayi in katsayilar:
        deger+=katsayi*x**indis
        indis+=1
    return deger
```

Alan hesabı modülü içeriği aşağıdaki gibidir;

```
# alan.py
"""Bu Modül çeşitli geometrik şekillerin alanını hesaplar."""
from math import sqrt
def kareninAlani(x):
    """bir kenarı bilinen karenin alanı"""
    return x*x
def dikdörtgeninAlani(x,y):
    """eni ve boyu bilinen dikdörtgenin alanı"""
    return x*y
def ucgeninAlani(x,y,z):
    """Üç kenarı bilinen üçgenin alanı (heron formülü)"""
    s= (x+y+z)/2
    return sqrt(s*(s-x)*(s-y)*(s-z))
```

Özel dosyanın (__init__.py) içeriği aşağıdaki gibidir;

```
from .polinom import polinomDegeri
from .alan import kareninAlani,dikdörtgeninAlani,ucgeninAlani
```

Paket içerisindeki alan hesabını kullanacak örnek kod aşağıdaki gibidir;

```
# alanOrnek.py
from hesap.alan import ucgeninAlani
print (" 5,6,7 Üçgenin alanı:", ucgeninAlani(5,6,7))
# 5,6,7 Üçgenin alanı: 14.696938456699069
```

Paket içerisindeki polinom hesabını kullanacak örnek kod aşağıdaki gibidir;

```
# polinomOrnek.py
from hesap.polinom import polinomDegeri
print ("x=1 için Polinom Değeri :", polinomDegeri(1,5.0,1.0)) # x=1, 5x**0+1x**1
#x=1 için Polinom Değeri : 6.0
```

Eğer bu paketi sistem genelinde kullanmak için **kurmak** (**setup**) istersek hesap klasörü ile aynı klasörde olacak şekilde **setup.py** dosyası hazırlarız.

D:\	→
└─ PythonKlasoru\	→ Python Betik Dosyalarının Bulunduğu Klasör
└─ hesap\	→ Paket Adını Belirleyen Klasör
└─ __init__.py	→ Paket Listesini İçeren Özel Dosya
└─ polinom.py	→ paket içindeki betik dosyası
└─ alan.py	→ paket içindeki bir başka betik dosyası
└─ alanOrnek.py	→ Paket dışındaki betik dosyası
└─ polinomOrnek.py	→ Paket dışındaki bir başka betik dosyası
└─ setup.py	→ Paketi kurulabilir yapmak için betik dosyası

Dosyanın içeriği aşağıdaki gibi olabilir;

```
#setup.py
from setuptools import setup
setup(name='hesap',
      version='0.1',
      description='Alan ve polinom hesabını içerir',
      url='#',
      author='İlhan Özkan',
      author_email='hoydabre@gmail.com',
      license='MIT',
      packages=['hesap'],
      zip_safe=False)
```

Yukarıdaki gibi önceden hazırlanmış Python paketlerini yüklemek için **PIP** (Package Installer for Python) aracı kullanılır. Bir paketi yüklemek için komut satırında aşağıdaki komut çalıştırılır;

```
C:\Users\ILHANOZKAN>pip install <paket adı>
// Ya da
C:\Users\ILHANOZKAN>python -m pip install <paket adı>
```

Hazırladığımız hesap paketini paket yükleme aracı kullanılarak kurulumu aşağıdaki şekilde yapılır;

```
D:\PythonKlasoru> pip install .
Processing d:\PythonKlasoru
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: hesap
  Building wheel for hesap (pyproject.toml) ... done
  Created wheel for hesap: filename=hesap-0.1-py3-none-any.whl size=2016
sha256=a6827110636b8bb072e3431f1ad7664ba56081bbaec495933802a065138c0b41
  Stored in directory: C:\Users\ILHANOZKAN\AppData\Local\Temp\pip-ephem-wheel-cache-
fzr6qquh\wheels\98\62\e3\ed2d7957ca1514c88218e667de8a81a5632b3b9b01640ecca1
Successfully built hesap
Installing collected packages: hesap
Successfully installed hesap-0.1

[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
D:\PythonKlasoru>
```

Aynı paketi kaldırmak istersek aşağıdaki komut çalıştırılabilir;

```
D:\PythonKlasoru> pip uninstall hesap
Found existing installation: hesap 0.1
Uninstalling hesap-0.1:
  Would remove:
    c:\users\ILHANOZKAN\appdata\local\programs\python\python313\lib\site-packages\hesap-
0.1.dist-info\*
    c:\users\ILHANOZKAN\appdata\local\programs\python\python313\lib\site-packages\hesap\*
Proceed (Y/n)?
Successfully uninstalled hesap-0.1
D:\PythonKlasoru>
```

Lamda İfadeleri

Lambda ifadeleri (lambda expression) anonim fonksiyonlardır, yani bir adları yoktur. Bildiğimiz gibi, `def` anahtar kelimesi Python diline normal bir fonksiyonu tanımlamak için kullanılır. Benzer şekilde, `lambda` anahtar kelimesi de anonim bir fonksiyonu tanımlamak için kullanılır.

Lamda ifadelerinin sözdizimi aşağıdaki şekildedir;

```
lambda argümanlar : ifade
```

Aşağıda **metin** (string) olarak argüman alan üç farklı lamda fonksiyonu tanımlanmıştır;

```
str = "Python Esnek Bir Dildir!"
küçüğeCevir = lambda s: s.upper()
print(küçüğeCevir(str)) #PYTHON ESNEK BIR DILDIR!
buyugeCevir = lambda s: s.lower()
print(buyugeCevir(str)) #python esnek bir dildir!
birlestir = lambda birinci,ikinci: birinci+" "+ikinci #iki parametrelili lamda
print(birlestir("İlhan","Özkan")) #İlhan Özkan
```

Bir lamda ifadesi `if` içerebilir. Buradaki kullanım `if` talimatından farklı olduğuna dikkat ediniz.

```
durum = lambda x: "Pozitif" if x > 0 else "Negatif" if x < 0 else "Sıfır"
print(durum(1)) #Pozitif
print(durum(-1)) #Negatif
print(durum(0)) #Sıfır
tekmiçiftmi=lambda x: "Çift" if x % 2 == 0 else "Tek"
print(tekmiçiftmi(2)) #Çift
print(tekmiçiftmi(1)) #Tek
```

Lamda ifadelerini Liste (list) başlığında anlatılan liste kavrama (list comprehension) ile birleştirmek mümkündür. Böylece dönüşümleri verilere özlü bir şekilde uygulayabiliriz.

```
liste = [1, 2, 3, 4, 5, 6]
onkat = [lambda arg=x: arg * 10 for x in liste]
for i in onkat:
    print(i(),end=' ')
# 10 20 30 40 50 60
print()
```

Bir lamda fonksiyonunda demet (tuple) kullanılarak birden fazla değer geri döndürülebilir;

```
hesaplama = lambda x, y: (x + y, x * y)
sonuclar = hesaplama(5, 6)
print(sonuclar) #(11,30)
```

Python dilindeki `filter()` fonksiyonu, argüman olarak bir fonksiyon ve bir liste alır. Bu fonksiyon listedeki uygun elemanlar için `True` döndürerek filtreleme sağlar.

```
liste = [1, 2, 3, 4, 5, 6]
ciftler = filter(lambda x: x % 2 == 0, liste) # İlk parametre lamda fonksiyonudur
print(type(ciftler)) #<class 'filter'>
print(list(ciftler)) #[2, 4, 6]
```

Python dilindeki `map()` fonksiyonu, bir fonksiyon ve bir listeyi argüman olarak alır. Fonksiyon, bir lambda fonksiyonuyla çağrılır ve her bir öğe için bu fonksiyon, lamda ile değiştirilmiş öğeleri içeren yeni bir liste döndürülür.

```
liste = [1, 2, 3, 4, 5, 6]
yuzkat = map(lambda x: x *100, liste)
print(type(yuzkat)) #<class 'map'>
print(yuzkat) #[100, 200, 300, 400, 500, 600]
```

Python dilindeki `reduce()` fonksiyonu, bir fonksiyon ve bir listeyi argüman olarak alır. Fonksiyon, bir yinelemeli fonksiyonla çağrılır ve yeni bir indirgenmiş tek sonuç döndürülür. Bu, yinelemeli fonksiyon çiftleri üzerinde tekrarlayan bir işlem gerçekleştirir. `reduce()` fonksiyonu `functools` modülüne aittir.

```
from functools import reduce
liste = [1, 2, 3, 4, 5, 6]
elemanlaritopla = reduce(lambda x,y: x+y, liste) #lamda fonksiyonu ardışık iki üyeyi toplar
print(type(elemanlaritopla)) #<class 'int'> #sonuç tek bir int değeridir.
print(elemanlaritopla) #21
```

Hem `lamda` hem de `def` kelimeleri, fonksiyonları tanımlamak için kullanılabilir, ancak farklı amaçlara hizmet ederler. `def` standart yeniden kullanılabilir (reusable) fonksiyonlar oluşturmak için kullanılırken,

lamda fonksiyonları çoğunlukla geçici olarak ihtiyaç duyulan kısa ve öz işlemleri gerçekleştiren anonim fonksiyonlar için kullanılır.